

To differentiate between these three quantifiers, consider the string *aaaaa*. The regular expression pattern *'a+a'* involves a greedy quantifier by default. This pattern will result in a greedy match of the whole string *aaaaa* because the pattern *'a+'* will match only the string *aaaaa*. Now consider the reluctant quantifier *'a+?a'*. This pattern will only result in a match for the string *aa* since the pattern *'a+?'* will only match the single character string *a*. Now let us see the effect of the Java specific possessive quantifier denoted by the pattern *'a++a'*. This pattern will not result in any match because the possessive quantifier behaves like a greedy quantifier, except that it is possessive. So, the pattern *'a++'* itself will possessively match the whole string *aaaaa*, and the last character *a* in the regular expression pattern *'a++a'* will not have a match. So, a possessive quantifier will match greedily and after a match it will never give away a character.

You can download and test the three example Java files *Greedy.java*, *Reluctant.java* and *Possessive.java* for a better understanding of these concepts. In Java, regular expression processing is enabled with the help of the package *java.util.regex*. This package was included in the Java Class Library (JCL) by J2SE 1.4 (JDK 1.4). So, if you are going to use regular expressions in Java, make sure that you have JDK 1.4 or later installed on your system. Execute the command:

```
java -version
```

... at the terminal to find the particular version of Java installed on your system. The later versions of Java have fixed many bugs and added support for features like named capture and Unicode based regular expression processing. There are also some third party packages that support regular expression processing in Java but our discussion strictly covers the classes offered by the package *java.util.regex*, which is standard and part of the JCL. The package *java.util.regex* offers two classes called *Pattern* and *Matcher* two classes called *Pattern* and *Matcher* that are used are used jointly for regular expression processing. The *Pattern* class enables us to define a regular expression pattern. The *Matcher* class helps us match a regular expression pattern with the contents of a string.

Java programs using regular expressions

Let us now execute and analyse a simple Java program using regular expressions. The code given below shows the program *Regex1.java*.

```
import java.util.regex.*;

class Regex1
{
    public static void main(String args[] )
```

```
{
    Pattern pat = Pattern.compile("Open Source");
    Matcher mat = pat.matcher("Magazine Open Source For
You");
    if(mat.matches( ))
    {
        System.out.println("Match from " + (mat.
start( )+1) + " to " + (mat.end( )));
    }
    else
    {
        System.out.println("No Match Found");
    }
}
```

Open a terminal in the same directory containing the file *Regex1.java* and execute the following commands to view the output:

```
javac Regex1.java and Java Regex1
```

You will be surprised to see the message *'No Match Found'* displayed in the terminal. Let us analyse the code in detail to understand the reason for this output. The first line of code:

```
'import java.util.regex.*;'
```

...imports the classes *Pattern* and *Matcher* from the package *java.util.regex*. The line of code:

```
'Pattern pat = Pattern.compile("Open Source");'
```

...generates the regular expression pattern with the help of the method *compile()* provided by the *Pattern* class. The *Pattern* object thus generated is stored in the object *pat*. A *PatternSyntaxException* is thrown if the regular expression syntax is invalid. The line of code:

```
'Matcher mat = pat.matcher("Magazine Open Source For You");'
```

...uses the *matcher()* method of *Pattern* class to generate a *Matcher* object, because the *Matcher* class does not have a constructor. The *Matcher* object thus generated is stored in the object *mat*. The line of code:

```
'if(mat.matches( ))'
```

...uses the method *matches()* provided by the class *Pattern* to perform a matching between the regular expression pattern *'Open Source'* and the string *'Magazine Open Source For You'*. The method *matches()* returns *True* if there is a match and returns *False* if there is no match. But the important thing to remember is that the method *matches()* returns *True* only if the